

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Semi-Supervised and Disentangled Causal Discovery for Analyzing Fault Propagation in Microservices

NOBUKAZU FUKUDA¹, HARUHISA NOZUE¹, HARUO OISHI¹, CHAO WU¹, SHINGO HORIUCHI¹, and KENICHI TAYAMA.¹

¹Access Network Service Systems Labs., NTT, Inc., Tokyo, Japan

Corresponding author: Nobukazu Fukuda (e-mail: nobukazu.fukuda@ntt.com).

ABSTRACT While troubleshooting fault propagation in a microservice-based system is labor-intensive for operators, it is critical to their decision-making process for maintaining availability and reliability. Efficient troubleshooting can significantly accelerate fault recovery and help strengthen preventive measures. Therefore, fault analysis techniques demand high interpretability of not only the fault location but also the fault propagation. However, existing works have mainly focused on developing root cause analysis, which cannot capture such structures among microservices. In this paper, we propose semi-supervised causal structure learning to capture the fault propagation across microservices from the time series data supported by continuous structure learning. We also introduce disentanglement learning to enhance the generalizability of out-of-distribution graphs of new faults. Extensive experiments on microservice-based system datasets demonstrate that the semi-supervised approach is effective, and the proposed model is superior to baseline methods.

INDEX TERMS AIOps, Microservice Systems, Causal Structure Learning, Semi-supervised Learning, Disentanglement Learning

I. INTRODUCTION

MICROSERVICE architecture has gained widespread popularity in a wide range of domains, driven by its distinct advantages [1]. Compared with a monolithic service, which provides all functions in a single binary, microservice-based architecture has advantages such as cross-team development, resource flexibility, cost-effectiveness, and rapid deployment [2], [3]. To overcome the limitations of software development of monolithic services, major cloud services have adopted large-scale microservice-based architectures [4]. Moreover, the significance of cloud-native networks powered by microservice-based systems is highlighted through enticing use cases such as network slicing for telecommunication service providers [5], [6]. With the proliferation of microservice-based systems, their operation and management have become emerging challenges that must be considered from several practical perspectives.

Microservice-based systems require continuous monitoring to maintain their robustness and stability and ensure reliable service delivery. Due to the large number of microservices deployed on a dynamic cloud infrastructure, the microservice-based architecture poses new challenges. To

effectively manage the complex distributed systems, machine learning techniques have been adopted for various diagnostic and optimization tasks, including anomaly detection in time series [7]–[9] and log data [10], [11], performance prediction [12], [13], and root cause analysis (RCA) [14], [15]. These monitoring and diagnostic techniques enable system operators to enhance the availability and reliability of their systems.

Recently, the concept of the causal structure between services or monitoring data has emerged as a new frontier for monitoring technology, primarily motivated by the need for interpretability [16]. Since system performance degradation is often unpredictable and difficult to trace, operators must invest significant effort in manual troubleshooting to fully comprehend fault conditions [17]. Causal graph-based visualization can greatly assist operators in achieving a comprehensive understanding of faults, playing a critical role in the decision-making process. Causal structure learning aims to explain the generative process of monitoring data by identifying causal relationships, such as in the form of directed acyclic graphs (DAGs) [18]. Consequently, the role of causal structures in distributed systems, particularly in explainability

for RCA, has been explored in numerous studies focusing on enhancing prediction accuracy and visualizing system behavior [19]–[27]. The construction of precise causal graphs is expected to significantly improve fault identification accuracy and increase troubleshooting efficiency. In particular, such insights enable operators to better understand system behavior during fault occurrences, including how faults propagate.

Despite recent advancements, several challenges remain in implementing causal structure-aware approaches for practical troubleshooting. Most causal discovery research relies on simulated evaluations [22], [23], [25] due to the scarcity and high cost of annotated datasets for causal structure learning [28]. Relying solely on simulated causal mechanisms limits their applicability to the complexities and diversity of real-world scenarios [21], [29], [30]. Furthermore, those methods focus exclusively on the accuracy of RCA without emphasizing the construction of precise causal graphs. As a result, their resulting graphs serve merely as intermediate representations, offering limited insights into fault propagation. To improve troubleshooting efficiency, it is essential to develop techniques capable of predicting highly accurate causal graphs in real-world distributed systems.

This paper seeks to investigate the potential efficacy of direct supervision to increase the effectiveness of causal structure learning employed in existing RCA frameworks. By accepting the annotation cost for a few cases and defining illustrative examples of causal graphs, we can offer valuable insights into the target system and the fault propagation. There are two primary challenges associated with this approach. **Challenge 1: Limited availability of gold labels due to high annotation costs.** Unlike the binary labels commonly used in existing studies, causal graphs exhibit greater structural complexity. Therefore, this challenge is to achieve highly accurate causal structure learning despite the scarcity of supervised data available for causal graphs. **Challenge 2: Risk of overlearning in direct supervision.** Dependence on supervised data introduces the risk of overfitting, where the model memorizes known fault features but fails to generalize to unseen fault scenarios. Consequently, this challenge is to ensure accurate causal graph estimation, even for faults not observed in annotated data.

We introduce a semi-supervised and disentanglement framework for causal structure learning on microservices. The proposed method constructs a causal graph from time series data employing gradient-based causal structure learning. Our framework first encodes the monitoring metrics into intermediate representations of service units. It then estimates edge probabilities between services to generate a causal graph that explains the fault propagation. These estimated probabilities are trained using supervised data to reflect the metric characteristics of the target system. In real-world environments, acquiring sufficient labeled data is often challenging, whereas large amounts of unlabeled data are readily available. Therefore, we jointly leverage a large amount of unlabeled data for causal structure discovery, while incorporating a limited amount of labeled data to enhance the

representation of the target system features within the causal structure. This semi-supervised approach improves the causal graph accuracy and effectively addresses the first challenge. Additionally, we introduce disentanglement learning, a technique widely studied in the computer vision domain [31]–[34], to mitigate out-of-distribution issues by decoupling fault-dependent features from causal graph generation. This approach ensures the generalizability of the causal graph and addresses the second challenge. The contributions of this paper are summarized as follows:

- We propose a novel framework for causal structure learning in microservices, leveraging time series data. Our approach enhances the accuracy and generalizability of causal graph construction.
- We conduct comprehensive experiments on datasets collected from a microservice-based benchmark. Experimental results demonstrate that the proposed method outperforms baseline approaches for causal structure learning.

The remainder of this paper is structured as follows. Section II provides a review of relevant literature and discusses the background of this research. Section III presents the preliminaries of gradient-based causal structure learning along with our problem setting. The proposed methodology is detailed in Section IV. Experimental settings and comprehensive results are described in Section V. Finally, Section VI concludes the paper with final remarks and directions for future research.

II. RELATED WORK

Causal structure learning discovers causal graphs from the observed data in the form of DAGs. It has successfully addressed various issues in numerous domains [18]. Constraint-based algorithms [35]–[37] construct causal graphs by performing independence tests that test the existence of edges connecting variable pairs. In particular, the PC algorithm [35] tests the independence of variable pairs to determine the causal skeleton and edge direction based on the predefined rules that the causal graph must satisfy. Since the introduction of score-based learning in DAGs [38], which provides efficient solutions, research has shifted from combinatorial optimization to continuous optimization [39]–[44]. By solving a continuous optimization problem with an acyclicity constraint, NOTEARS [38] achieves a global optimal solution for causal structure learning of DAGs. Instead of constrained optimization, GOLEM [42] employs a likelihood-based objective function incorporating soft sparsity and DAG constraints to identify linear relationships. While these methods offer flexibility, they rely solely on node variables and exhibit sub-optimal performance when applied to real-world data from distributed systems. To address this limitation, our approach enhances causal structure learning for actual monitoring data by utilizing a semi-supervised learning framework.

RCA for fault analysis of complex distributed systems has been extensively studied [19], [45], [46]. In response

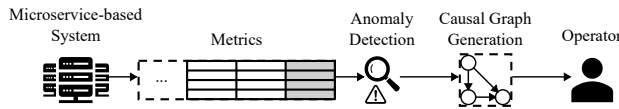


FIGURE 1: The illustration of the fault propagation analysis workflow.

to the growing need for interpretability, causal-structure-aware RCA has attracted significant attention in recent years [19], [21], [23], [26], [27], [47]. Using the PC algorithm, Kobayashi *et al.* [20] implemented architecture-aware heuristics to construct causal graphs from log data. Meng *et al.* [48] developed a variant of a constraint-based algorithm to generate a causal graph and designed a random walk on the graph to rank potential root causes. Gan *et al.* [49] proposed a causal Bayesian network incorporating domain knowledge of microservice-based systems and causal assumptions. Additionally, Chakraborty *et al.* [25] detected causal graphs using performance metric variations informed by system architecture. The assumptions underlying causal relationships between metrics are also widely discussed in [22]. Li *et al.* [24] trained a supervised deep learning model that references a fault dependency graph to identify a suspect service from monitoring metrics. These interpretable RCA methods are supported in their validity by the causal structure learning frameworks. However, real-world applications reveal that existing RCA methods often suffer from inaccurate performance and poor interpretability [30]. To address these challenges, we aim to enhance causal structure learning for interpretable RCA methods, improving their applicability to fault analysis. Our approach can be integrated into causal graph search components of existing causal-structure-aware RCA frameworks.

To improve performance with a limited set of labeled samples, several learning paradigms, including semi-supervised learning, have garnered significant interest in practical implementations of deep learning approaches [50]. It is expensive to annotate the monitoring data of running systems and to collect anomaly data that covers a wide range of faults. Consequently, semi-supervised learning, along with positive and unlabeled learning, has been introduced to address operational challenges of distributed systems, such as intrusion detection [51], log anomaly detection [52], [53], and trace anomaly detection [54]. These methods rely on input data format and target specific tasks, enhancing performance by leveraging additional data sources and domain knowledge. To further enhance causal inference in fault detection, we propose a practical semi-supervised learning approach that integrates unsupervised causal structure learning. Our framework can be readily implemented in conventional microservice-based systems, offering improved adaptability and robustness in real-world services.

III. PROBLEM SETTINGS AND PRELIMINARIES

This section clarifies the monitoring setting and the fault-

propagation graph estimation problem considered in this paper. As illustrated in Fig. 1, we observe streaming multivariate time series metrics from a microservice-based system. When a fault occurs, anomalous fluctuations may propagate across services through their interactions. Our goal is to summarize such fault-induced propagation within each time window as a service-level DAG, where each node corresponds to a microservice, and each directed edge indicates the direction of anomaly propagation. This graph can be used as the causal-structure component of causal-structure-aware RCA pipelines; other modules, such as anomaly detection and root cause ranking, can be integrated on top of the estimated graph. Because accurate propagation graphs are expensive to annotate at scale, we consider a semi-supervised setting where only a small subset of time windows is associated with partial graph annotations.

A. PROBLEM SETTINGS

Metrics consist of streaming multivariate time series data that provide system status insights to operators. These time series data are collected via monitoring techniques from various system components and organized into a matrix representation. For instance, CPU utilization, memory consumption, disk I/O usage, network throughput, and other key metrics are recorded from physical machines, virtual machines, containers, pods, and other infrastructure elements. Anomalous fluctuations in these metrics, particularly near suspicious components, can indicate potential fault occurrences. By analyzing these variations, operators can diagnose system failures (e.g., CPU depletion, memory leaks) and subtle performance degradation, enabling proactive maintenance and rapid fault resolution. The causal structure of microservices typically represents the services or metrics as nodes, and this paper adopts a service-level causal structure following existing causal-aware RCAs [15], [55], [56]. While causal structures that model metrics as nodes offer rich granularity, they tend to be overly complex for operators to interpret efficiently, and intermittently collected monitoring data grows explosively as systems operate over extended periods.

We define our problem settings by considering the streaming monitoring data of the target microservices within specific time windows. These metrics exhibit significant variations over time in response to fault occurrences, and depending on the dynamic relationships between the services, such fluctuations in a metric will propagate to other metrics. To capture this propagation structure, we identify the structure of the propagation relations when a fault occurs and construct a service-level causal graph. In particular, we model the propagation process of observed values using the additive noise model and construct a causal graph that represents the generative process of observed data under a fault. Note that the proposed model can be applied to granular-level causal structures without modifying its architecture, and we also restrict candidate edges of the estimated propagation graph to those consistent with observed or known service invocations (details are presented in Section IV).

Furthermore, we introduce a semi-supervised setting into the causal structure learning of microservices. Due to the vast number of metrics and the complexity of fault propagation, manually annotating highly accurate and consistent causal graphs is impractical, and unlike conventional RCAs that identify faults, ground-truth causal graphs are unavailable because of high annotation costs. Thus, we leverage a limited set of supervised causal graphs as clues for learning complex causal structures. Our objective is to infer complex causal relationships from metrics and partial graphs within a specific time window in microservices experiencing faults, and subsequently predict the DAG between microservices from only the metrics collected within a new time window.

B. PRELIMINARIES

To connect the above problem to standard causal discovery formulations, we provide an overview of causal structure learning. Let $\mathbf{Z} = \{z_1, z_2, \dots, z_S\} \in \mathbb{R}^{S \times T}$ be multivariate time series data, where $z_i \in \mathbb{R}^T$ denotes the time series associated with i -th node. A causal graph in the form of a DAG with S time series as vertices defines the generative process of $\mathbf{Z}$. With $z_{pa(i)}$ as the parent set connected to z_i in G , the joint probability $P(\mathbf{Z})$ can be denoted as follows [57].

$$P(\mathbf{Z}) = \prod_i P(z_i | z_{pa(i)}). \quad (1)$$

With F_i as a vector function expressing the generative process of z_i and ϵ_i as the noise of z_i , the conditional distribution of z_i under the additive noise model [58] can be written as follows.

$$z_i = F_i(z_{pa(i)}) + \epsilon_i. \quad (2)$$

Then, letting $\mathbf{A} \in \mathbb{R}^{S \times S}$ be the weighted adjacency matrix of the DAG G , a linear structure equation model is derived. This linear-SEM assumption is widely used in continuous optimization-based causal discovery in RCAs [26], [27].

$$\mathbf{Z} = \mathbf{AZ} + \boldsymbol{\epsilon}. \quad (3)$$

Here, the following constraint [38] is introduced to ensure the acyclicity of G . This constraint calculates the matrix exponent of the Hadamard product to count paths of arbitrary length between any two nodes in G , ensuring that there are no directed cycles.

$$\text{tr}(e^{\mathbf{A} \circ \mathbf{A}}) - S = 0. \quad (4)$$

Here, $\text{tr}(\cdot)$ denotes the trace of a matrix, and $\circ$ denotes the Hadamard product of two matrices. Furthermore, an alternative constraint is introduced to ease the computation of the matrix exponent [39].

$$\text{tr}((\mathbf{I} + \alpha \mathbf{A} \circ \mathbf{A})^S) - S = 0. \quad (5)$$

The hyperparameter α is intended to alleviate the difficulty in computing the exponents that depend on the magnitude of the eigenvalues of $\mathbf{A} \circ \mathbf{A}$. With these functional continuous constraints, we can ensure the acyclicity of G when optimizing loss functions. The causal directions are guided by the data

signals, while the SEM-based structural regularizer primarily encourages the learned graph to be sparse and acyclic. Based on the above problem setting and preliminaries, Section IV presents the proposed framework for estimating the propagation graph.

IV. METHODOLOGY

To address the challenges outlined in this paper, we introduce a semi-supervised learning framework for constructing causal graphs of faults in microservices. This section provides an overview of our approach, followed by a detailed description of each component of the proposed method.

A. OVERVIEW

Fig. 1 shows the workflow of the fault propagation analysis. The proposed method processes time series metrics collected by monitoring agents, dividing them into batches within a pre-defined time window. Each batch of multivariate time series data serves as input for the model, which learns causal relationships between metric fluctuations offline using a partially labeled causal graph. During the prediction phase, the model utilizes trained encoders to process the metrics and employs binary classifiers to predict the edges of the causal graph. This inference step does not rely on labeled data to generate causal graphs directly from raw metrics. Compared to the time interval for monitoring data collection, the proposed method achieves near real-time graph generation, allowing immediate reporting to the operator. While the primary focus is on causal graph estimation, other existing modules, such as log anomaly detection and RCA, can be integrated to improve troubleshooting effectiveness.

Fig. 2 provides an overview of the proposed architecture. Our model consists mainly of the following components.

- 1) **Causal Structure Learning** (Fig. 3), which applies a linear causal structure to the metrics using the causal graph estimated by the encoder.
- 2) **Disentanglement Learning** (Fig. 4), which decouples fault-specific features from the causal graph generation process of the encoder.

B. CAUSAL STRUCTURE LEARNING

During offline training, for each time window, the metrics encoder produces service-level representations, and the graph generation module predicts edge probabilities under the call-graph constraint. When partial edge annotations are available, we additionally apply a supervised edge loss to calibrate the predicted edge existence. Regardless of label availability, we optimize the graph autoencoder reconstruction objective together with an acyclicity regularizer to leverage unlabeled windows. During online inference, the trained encoder and graph generation module take only streaming metrics as input and output a service-level directed graph, without requiring any graph annotations or fault labels.

Metrics Encoder. We aggregate the metrics collected from the target services into the service-level representation. A

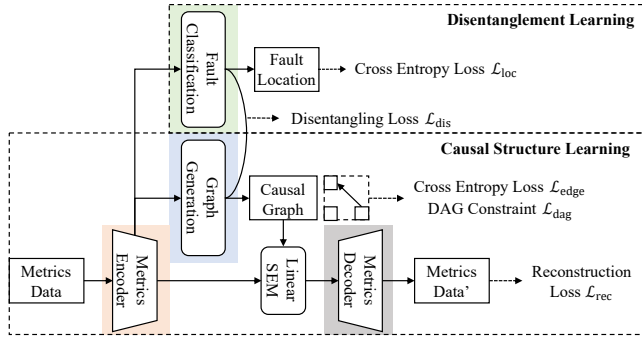


FIGURE 2: The architecture of the proposed model. Causal structure learning aims to discover the causal graph of input metrics. Disentanglement learning aims to decouple the fault-dependent features from the metrics encoder.

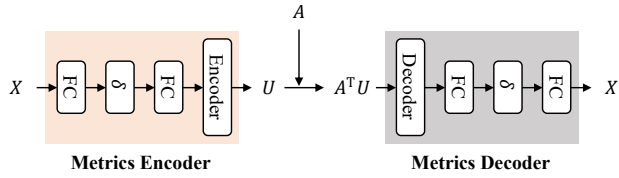


FIGURE 3: The architecture of encoder-decoder causal structure learning. The background colors correspond to the components in Fig. 2.

batch of a time window contains a series of metrics $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_M\} \in \mathbb{R}^{M \times T}$ ($\mathbf{x}_i \in \mathbb{R}^T$). To construct a service-level causal graph, we aggregate these M metrics into S vector variables. Specifically, we apply deep nonlinear functions to address the complex relationships between services over time. First, we map $\mathbf{X}$ into service-level representations $\mathbf{U} = \{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_S\} \in \mathbb{R}^{S \times d}$ ($\mathbf{u}_i \in \mathbb{R}^d$).

$$\mathbf{U}^T = f_1(\mathbf{W}_2 \text{ReLU}(\mathbf{X}^T \mathbf{W}_1)). \quad (6)$$

Here, $\mathbf{W}_1 \in \mathbb{R}^{M \times S}$, $\mathbf{W}_2 \in \mathbb{R}^{d \times T}$ are the parameter matrices. $\text{ReLU}(\cdot)$ denotes the activation function of rectified linear unit (ReLU) (δ in Fig. 2), and $f_1(\cdot)$ denotes a nonlinear function. To model complex relationships in multivariate time series data, we adopt the Transformer [59] encoder as $f_1(\cdot)$. Note that the resulting service representations capture intra-window temporal lags with the encoder, rather than instantaneous causality. Furthermore, the parameter matrix $\mathbf{W}_1$ of the linear layer serves as an aggregator for all the metrics and is also utilized in the preprocessing step of the baseline methods in Section V.

Graph Generation. After obtaining the service-level representations $\mathbf{U}$, we predict the edges to construct a causal graph. Let $\mathbf{A}$ denote an adjacency matrix of a causal graph G . Then, the edge probability of G and the weights of $\mathbf{A}$ are computed by the following process. We first concatenate $\mathbf{u}_i$ and $\mathbf{u}_j$ of each service corresponding to an edge from service i to j . Here, we utilize prior knowledge of the call graph and enumerate only the edges where possible invocations occurred. After that, we compute the edge weights of

the causal graph. Since we need the edge weights and edge existence probabilities, respectively, we compute the positive and negative scalar values of edge weights as follows.

$$E_{ij}^P = \mathbf{W}_P^{(l_i)} \text{ReLU}(\dots \text{ReLU}(\mathbf{W}_P^{(1)} \text{Concat}(\mathbf{u}_i, \mathbf{u}_j))). \quad (7)$$

$$E_{ij}^M = \mathbf{W}_M^{(l_i)} \text{ReLU}(\dots \text{ReLU}(\mathbf{W}_M^{(1)} \text{Concat}(\mathbf{u}_i, \mathbf{u}_j))). \quad (8)$$

Here, $\mathbf{W}_P^{(l_i)}$, $\mathbf{W}_M^{(l_i)}$ are the parameter matrices of the MLP. Also, Concat represents the concatenation operation of vectors. Then, we compute the difference of E_{ij}^P and E_{ij}^M to obtain the weight of the edge (i, j) of the adjacency matrix $\mathbf{A}$.

$$(\mathbf{A})_{ij} = \text{ReLU}(E_{ij}^P) - \text{ReLU}(E_{ij}^M). \quad (9)$$

When a positive relation is inferred from the monitoring metrics, the corresponding positive weight increases, while the negative weight remains low. As a result, the first term in (9) attains a non-zero value, whereas the second term becomes zero. Since the edge weights can be negative, it is essential to compute E_{ij}^M to ensure accurate computation of the edge weights.

Given some labeled causal graphs, we can train the edge probabilities of the graph directly. The edge of G exists when the corresponding weight of $\mathbf{A}$ attains a significant value. We compute the edge existence probability E_{ij} and train the following cross-entropy loss $\mathcal{L}_{\text{edge}}$ with the gold labels $\hat{E}_{ij} \in \{0, 1\}$, where $|E|$ is the number of candidate edges.

$$E_{ij} = \text{Sigmoid}(\max(E_{ij}^P, E_{ij}^M)). \quad (10)$$

$$\mathcal{L}_{\text{edge}} = -\frac{1}{|E|} \sum_{(i,j) \in E} (\hat{E}_{ij} \log(E_{ij}) + (1 - \hat{E}_{ij}) \log(1 - E_{ij})). \quad (11)$$

Training the encoder to construct the causal graphs via annotated labels enhances the adaptability of the model, enabling accurate prediction of actual monitoring data. To refine the model parameters, the loss function is optimized whenever labeled data is available within a batch.

Metrics Decoder. After obtaining the adjacency matrix $\mathbf{A}$ of the graph, the model learns the causal structures using the unlabeled metrics $\mathbf{X}$. Following common causal structure learning architectures, such as those proposed in previous studies [27], [39], we employ a graph autoencoder incorporating the causal graph G shown in Fig. 3. The graph autoencoder consists of a metrics encoder and a decoder. The decoder, a nonlinear function symmetric to the encoder, reconstructs the metrics from the intermediate representations $\mathbf{U} \in \mathbb{R}^{S \times d}$. We optimize the following reconstruction loss $\mathcal{L}_{\text{rec}}$ and update the parameters of the graph autoencoder accordingly.

$$\mathcal{L}_{\text{rec}} = \|\mathbf{X} - \text{ReLU}(\mathbf{W}_3 f_2(\mathbf{A}^T \mathbf{U}) \mathbf{W}_4)\|^2. \quad (12)$$

Here, $\mathbf{W}_3 \in \mathbb{R}^{M \times S}$, $\mathbf{W}_4 \in \mathbb{R}^{d \times T}$ are the parameter matrices, and $f_2(\cdot)$ is a nonlinear function and adopts a symmetric architecture with the metrics encoder. The model captures relevant features from the metrics and learns the causal graph G to uncover the generative process of $\mathbf{X}$. Our approach constructs causal graphs within a causal structure learning

framework, leveraging unlabeled metrics, which are more readily available than labeled causal graphs.

DAG Constraint. Generating the causal graphs only through the graph autoencoder may result in dense and cyclical graphs. To ensure the acyclicity of the causal graph, we compute the following equality constraints [39] of the adjacency matrix A .

$$h(A) = \text{tr}((I + \alpha A \circ A)^S) - S. \quad (13)$$

Here, the number of services S is also the size of A . As stated in Section III, the graph is acyclic if and only if this function satisfies $h(A) = 0$. This constraint is integrated into the loss function $\mathcal{L}_{\text{dag}}$ during the optimization process.

C. DISENTANGLEMENT LEARNING

The proposed model learns edge probabilities directly from the labeled batches, ensuring high adaptability. However, it may overfit fault locations already learned through supervised labels. To mitigate this issue, we introduce disentanglement learning in the metrics encoder of the proposed model. Disentanglement learning of representations attempts to extract and decouple the underlying information embedded in the target data. This approach has been explored through adversarial learning [31] and independent representation embedding [26]. It has been widely applied in various areas, most notably in domain adaptation in the computer vision field [32], [33]. By introducing disentanglement learning into causal structure learning, the model can obtain invariant features for fault locations. Constructing a causal graph with fault-location invariant information enhances the generalizability of unseen data, thereby improving robustness to out-of-distribution scenarios. We introduce the disentangling loss [34] for feature decoupling within the proposed model.

As shown in Fig. 4, we jointly predict the causal graph (Graph Generation) and classify the fault (Fault Classification). We first obtain the service-level representations $U' \in \mathbb{R}^{S \times d}$ by encoding the metrics with the same parameter matrices W_1, W_2 and another Transformer encoder different from f_1 . Then, we employ fully connected layers to convert U' to the probabilities F_i for each fault. When labeled data are available, we optimize the cross-entropy loss $\mathcal{L}_{\text{loc}}$ of the estimated fault probabilities. Then, we introduce $\mathcal{L}_{\text{dis}}$ to discourage redundant alignment between the two vectors $u_s \in \mathbb{R}^d$ and $u'_s \in \mathbb{R}^d$.

$$\mathcal{L}_{\text{dis}} = \frac{1}{S} \sum_s \text{ReLU}(\text{Cosine}(u_s, u'_s)). \quad (14)$$

Here, $\text{Cosine}(\cdot, \cdot)$ represents the cosine similarity between the two vectors. This one-sided cosine penalty suppresses positive alignment between the two embeddings. This asymmetric constraint is used as a lightweight regularizer to mitigate feature/identity leakage while avoiding over-regularization of task-relevant representations [60], [61]. Symmetric variants such as $|\text{Cosine}(\cdot, \cdot)|^2$ are reasonable alternatives for stricter decorrelation with potentially stronger impact on reconstruction. The disentangling loss decouples the two features to

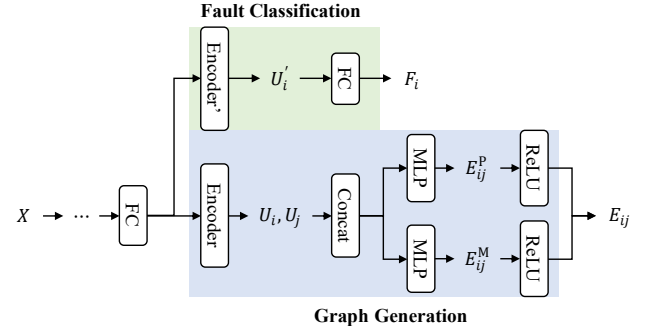


FIGURE 4: Disentanglement learning. The colors of the background correspond to the components in Fig. 2.

estimate the causal graph and classify the fault. This trick allows our model to learn the fault-invariant causal structure, enhancing the robustness to unseen fault locations.

D. OPTIMIZATION

Finally, we optimize all the preceding loss functions jointly. Specifically, the objective function of the unconstrained continuous optimization problem is as follows.

$$\mathcal{L} = \mathcal{L}_{\text{edge}} + \mathcal{L}_{\text{rec}} + \lambda \mathcal{L}_{\text{dag}} + \mathcal{L}_{\text{loc}} + \mathcal{L}_{\text{dis}}. \quad (15)$$

Here, λ controls the penalized degree of regularization terms. Note that loss terms $\mathcal{L}_{\text{edge}}$ and $\mathcal{L}_{\text{loc}}$ requiring labeled data are computed only when labeled data are available. Even when the edge labels or fault labels are unknown, the model is updated through the remaining loss terms in an unsupervised manner. When predicting causal graphs from streaming monitoring data, we can construct causal graphs solely from unlabeled metrics through the metrics encoder without computing all the loss terms.

Regarding computational complexity, in each training step, our method computes per-edge scores, yielding $O(|E|)$ with fixed depth. In practical microservice deployments, the call graph is often sparse, and thus $|E|$ is often close to linear in S . The graph autoencoder performs multiplications such as $A^T U$, which costs $O(S^2)$, and the Transformer-based encoder/decoder also incurs $O(S^2)$ complexity due to self-attention per training step. By contrast, the acyclicity penalty in (13) requires matrix operations on $S \times S$ matrices and scales as $O(S^3)$ [38]. As S increases, this cubic evaluation typically becomes the dominant training cost. During inference, it constructs causal graphs from unlabeled metrics via the metrics encoder and the per-edge scoring module without evaluating the training losses, including the acyclicity penalty. Therefore, the inference cost is mainly $O(S^2)$ from the Transformer-based encoder and $O(|E|)$ from per-edge scoring.

V. EXPERIMENTS

A. EXPERIMENTAL SETUP

For the DAG constraint in (13), we set $\alpha = 1/S$, where S is the number of services. We use AdamW with a learning rate

of 10^{-3} to optimize (15), and set $\lambda = 10^{-3}$. The Transformer encoders in the metrics encoder and decoder use one layer, 8 heads, and a hidden dimension of 128. We implement the proposed method with Python 3.11.4 and PyTorch 2.0.1 and run it on a system with an Intel Core i7-10610U 2.30 GHz. We briefly describe our datasets, the evaluation measures, and the baseline methods.

1) Datasets

We evaluate two datasets, $\mathcal{A}$ and $\mathcal{B}$, built from two microservice-based systems with both service-level metrics and distributed traces. Dataset $\mathcal{A}$ is collected from the OpenTelemetry-Demo v1.0.0¹, which consists of 12 microservices and a *loadgenerator* service that applies workload to the others. All metrics of each microservice are scraped using Prometheus². In addition, we collect all inter-service traces using Jaeger³, where distributed traces are recorded across services communicating via gRPC/HTTP. Dataset $\mathcal{B}$ is built from an existing dataset [62] collected from TrainTicket [63], which consists of 41 microservices. This dataset provides time-synchronized service-level metrics recorded at each microservice and the corresponding trace data captured concurrently. Dataset $\mathcal{A}$ yields 1,025 metric time series with non-trivial variations, whereas Dataset $\mathcal{B}$ includes a preprocessed subset of 351 selected metrics.

To analyze anomalous system behaviors, we use CPU-hog fault injection data in both datasets. For Dataset $\mathcal{A}$, CPU-hog faults are injected into 12 services (excluding *loadgenerator*), yielding experiments on 12 different fault locations. Each fault is injected for 600 seconds with sufficient intervals between injections, and monitoring data is collected during both fault injection and normal operating states. For Dataset $\mathcal{B}$, CPU-hog faults are injected into four services, and we use three runs per service (i.e., 12 runs in total). We segment the collected data into batches with a 180-second window, yielding 417 batches for Dataset $\mathcal{A}$ and 348 batches for $\mathcal{B}$, where each batch contains paired metrics and traces. Each dataset is partitioned into training, development, and evaluation subsets without duplication, where the development and evaluation sets account for 10% of the total batches each. The training subset is used for semi-supervised learning. To further evaluate generalization to unseen fault locations, we additionally construct an evaluation setting by holding out one fault location for evaluation, while using the remaining locations for training and development.

The collected traces are used to generate ground-truth causal graphs of fault propagation for numerical evaluation. A trace consists of a series of spans with the same traceID originating from a user request [56], and each span corresponds to an invocation between services and contains invocation information such as service names, operation names, and durations, where an operation is a unit of action within a service.

For each pair of services and operations, we assume that span durations under normal conditions follow the same distribution. We identify anomalous invocations by marking a span as anomalous when its duration exceeds the 75th percentile of the corresponding normal-state distribution, and we then list such anomalous invocations from the collected traces. This percentile-based criterion is analogous to the n -sigma heuristic adopted in prior work [19], [22], in that both define anomalies via distribution-based thresholding of deviations from normal behavior. Finally, we construct a ground-truth causal graph by connecting anomalous invocations starting from the fault-injected service. Here, we assume that metric fluctuations propagate along the service call graph during fault injection, and that deviations from normal conditions during fault injection are attributable to the injected fault. While this study uses the systematically created causal graphs for training and evaluation, the proposed method can also work with human-annotated causal graphs of service faults.

2) Evaluation Measures

We evaluate the proposed method based on these widely used evaluation measures as in [25], [30], [37].

- **Structural Hamming Distance (SHD)** [64] evaluates the number of changes required for the generated graph compared with the ground truth. To compare between graphs of different sizes, we report normalized SHD, dividing SHD by the number of edges in the ground-truth graph as in [65]. Compared with the ground-truth graph with T edges, let E be the number of extra edges, M the number of missing edges, and R the number of reversed edges. Then, normalized SHD can be denoted as follows, where a lower value indicates a better performance.

$$\text{normalized SHD} = \frac{E + M + R}{T}. \quad (16)$$

- **F1** denotes the correctness of the position and orientation of the edges. Comparing the generated graph and the ground truth, let TP be the number of edges with matching directions and O the number of edges in the generated graph. Then, precision, recall, and F1 can be denoted as follows, where a higher value indicates a better performance.

$$\begin{aligned} \text{Recall} &= \frac{TP}{T}, \text{ Precision} = \frac{TP}{O}, \\ \text{F1} &= \frac{2 \text{ Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}}. \end{aligned} \quad (17)$$

3) Baseline Methods

We compare our proposed framework with the following causal structure learning methods.

- **GrangerMV** [66] is a multivariate extension of Granger causality that fits vector autoregressive models and quantifies directed influence between variable groups.
- **PC** [35] is a constraint-based method that examines the existence of edges in the causal graph by conducting independence tests for each pair of time series data.

¹<https://opentelemetry.io/docs/demo/>

²<https://prometheus.io/>

³<https://www.jaegertracing.io/>

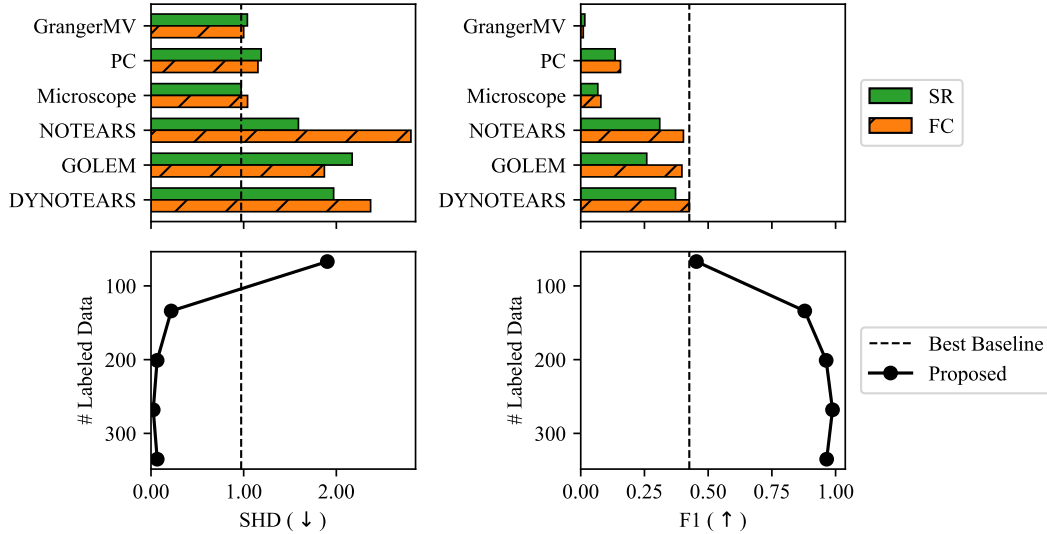


FIGURE 5: Overall performance on Dataset A: comparison between the baseline methods and the proposed method. The top row reports baseline results with two metric aggregation schemes (SR and FC), and the bottom row reports the proposed method with varying numbers of labeled batches. The vertical dashed line indicates the best baseline performance for each evaluation metric.

- **Microscope** [19] builds a causality graph from runtime observations, and then traverses the graph from an abnormal front-end to rank likely root cause candidates. Its causal discovery component is implemented as a parallelized variant of the PC algorithm.
- **NOTEARS** [38] is a gradient-based method that formulates structure learning as a continuous optimization problem and employs L-BFGS [67] to efficiently optimize a causal graph that explains the observed data.
- **GOLEM** [42] is an extension of NOTEARS, which optimizes a score function with continuously relaxed DAG constraints.
- **DYNOTEARS** [41] is a score-based dynamic Bayesian network that assumes a structural vector autoregressive model and jointly estimates contemporaneous and time-lagged dependencies.

As with the proposed model, all the baselines also utilize prior knowledge about the call graph for a fair comparison. Specifically, we restrict the construction of the causal graph for PC and mask the edge parameters for NOTEARS and GOLEM. While the proposed model can generate multiple causal graphs in a batch, the baseline methods sequentially explore individual causal graphs. The baseline approaches are applied to each batch of evaluation data, generating causal graphs one by one.

To align the granularity of the graph for each method, metric aggregation is employed to reduce complexity and enhance interpretability. The following approaches are used with the baseline methods.

- **SR** employs the Spectral Residual (SR) algorithm [8] to calculate the anomaly score of the time series data.

This approach represents the time series collected from a service as the most anomalous one within the service.

- **FC** leverages a fully connected aggregation layer that has the same architecture as the proposed model. To isolate the effect of the subsequent causal structure learning step, we set this aggregation layer with the parameters trained with all the labeled data. Therefore, it serves as an upper-bound aggregation baseline, providing the best-case aggregated representations for assessing the causal graph construction methods under a matched aggregation scheme.

B. RESULTS

We describe the experimental results to compare the proposed method with the baseline methods and demonstrate its effectiveness.

1) Overall Performance

Figs. 5 and 6 present the overall performance on Datasets A and B, respectively. In each figure, the upper bar plots illustrate the baseline results with the two metric aggregation schemes, while the lower line plots depict the performance of the proposed method across different numbers of labeled data. Here, the number of batches of labeled data used for cross-entropy loss computation varies from approximately 60 to 330, while unlabeled metrics from all batches are utilized in all cases. A lower SHD value and a higher F1 score indicate superior model performance. In each plot, the dashed vertical line represents the best baseline performance.

Overall, the proposed method improves as the number of labeled batches increases and surpasses the best baseline once a moderate number of labeled batches is available

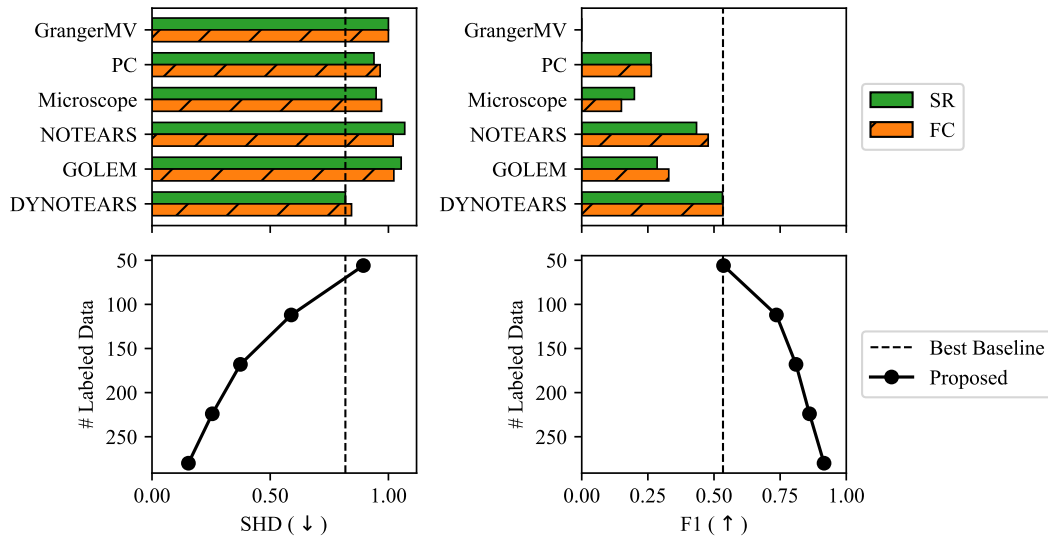


FIGURE 6: Overall performance on Dataset $\mathcal{B}$: comparison between the baseline methods and the proposed method. The top row reports baseline results with two metric aggregation schemes (SR and FC), and the bottom row reports the proposed method with varying numbers of labeled batches. The vertical dashed line indicates the best baseline performance for each evaluation metric.

(e.g., beyond roughly 100 labeled batches). Additionally, the comparison of F1 values shows that the proposed method matches or outperforms all baseline methods, even with only about 60 labeled data points. As observed in the comparison with the baseline methods with FC that use the same metric aggregation, the proposed method is superior in terms of causal structure learning alone. Consequently, the proposed method is more likely to generate a correct causal graph thanks to the semi-supervised learning framework. These results emphasize the effectiveness of direct supervision in causal structure learning, surpassing unsupervised baseline methods in performance.

In addition, with more than 200 labeled datasets, the proposed method can successfully predict the correct graph for nearly all batches. However, the performances of the baseline methods are inconsistent in some cases. Specifically, DYNOTEARS+FC achieves the best F1 score for both datasets, whereas Microscope+SR demonstrates the best SHD score for Dataset $\mathcal{A}$. This discrepancy can be attributed to different error profiles. Specifically, PC tends to output sparse graphs and may miss true edges, whereas optimization-based methods (e.g., NOTEARS and GOLEM) often infer denser graphs with more false positives. Thus, the experimental results demonstrate that the proposed semi-supervised causal structure learning is more effective than the baseline methods.

2) Influence of Reconstruction Loss

To investigate the advantage of semi-supervised learning over supervised learning alone, we compare the proposed method with and without learning the reconstruction loss for Dataset $\mathcal{A}$ in Fig. 7. The line plot in the figure depicts the performance of the proposed method using various num-

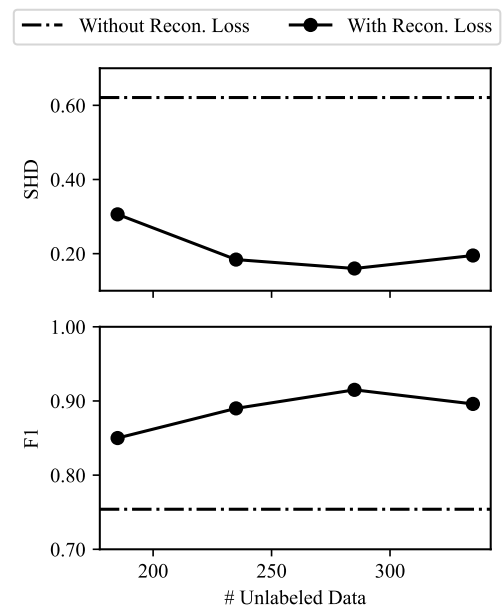


FIGURE 7: Influence of reconstruction loss: comparison results of different unlabeled data volumes. The line plots indicate the proposed method, and the dotted horizontal lines indicate the proposed method without the reconstruction loss.

bers of unlabeled data and a fixed number of 130 labeled data. The horizontal dotted lines indicate the performance of the proposed model without the reconstruction loss. Both evaluation measures demonstrate that performance improves as the number of unlabeled data increases. By leveraging readily available unlabeled data via the reconstruction loss,

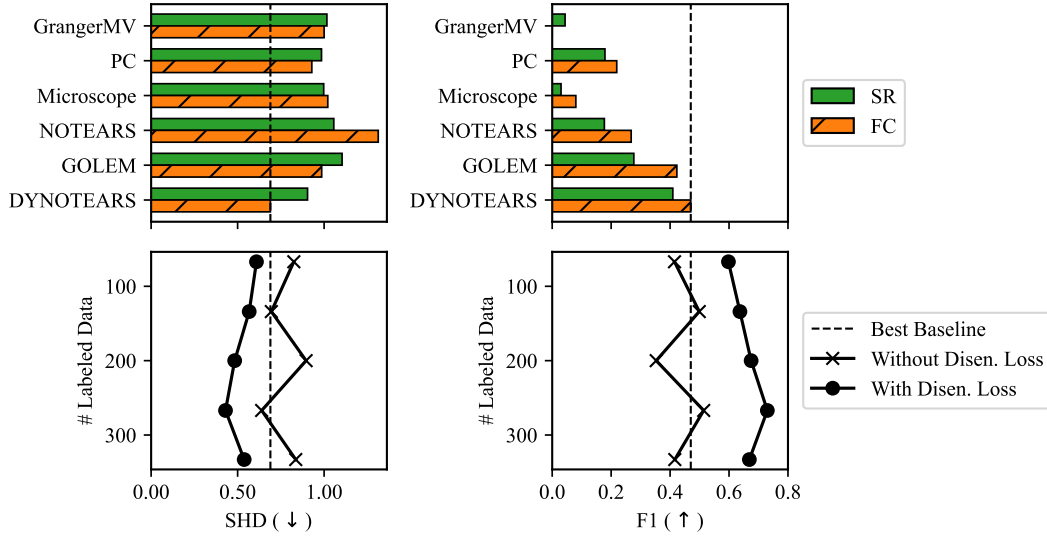


FIGURE 8: Influence of disentanglement learning: a study of the impact of the disentanglement learning. The bar plots indicate the baseline methods, and the horizontal dotted lines indicate the best ones. The line plots indicate the proposed method with and without the disentangling loss.

the proposed method enhances its ability to capture metric features through unsupervised structure learning. Thus, this experiment shows that the proposed method can effectively reduce the costs of the labeled data for semi-supervised causal structure learning. However, performance gains will be limited unless a sufficiently large amount of data is available. Notably, performance degradation is observed at the most significant number of unlabeled data, as the additional data do not always contribute positively to inference accuracy in evaluation scenarios. Furthermore, compared to Fig. 5, the performance gain from unlabeled data remains small when a sufficient amount of labeled data is available.

3) Influence of Disentanglement Learning

To investigate the generalizability of the proposed method, we compare its performance with and without the disentangling loss in Fig. 8. In this experiment, fault locations included in the evaluation data are excluded from the training data, which differs from the experiment in Fig. 5, to evaluate the performance on unseen fault locations. The upper bar plots report the baseline performance, while the lower line plots depict the proposed method with and without the disentangling loss as the number of labeled data increases. In each line plot, the vertical dashed line indicates the best baseline performance. As shown in the figure, the proposed method consistently outperforms baseline methods for unseen fault locations, while its performance is lower than that in the in-distribution setting (Fig. 5). Furthermore, the disentangling loss enhances the performance of the proposed method as the number of labeled data points increases. This is because the disentangling loss removes fault-specific features from the intermediate representations, allowing the encoder to capture fault-invariant features. This experiment demonstrates the ad-

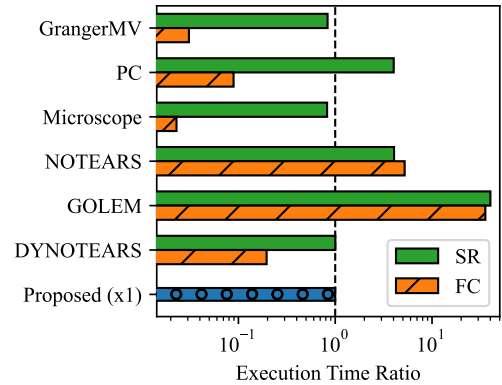


FIGURE 9: Time cost analysis: comparison of the time costs for the different methods.

vantage over unsupervised approaches, which improves generalization performance for unseen out-of-distribution fault locations by leveraging labeled data.

4) Time Cost Analysis

Fig. 9 presents a comparison of the inference time between the proposed method and the baseline methods for Dataset $\mathcal{A}$. For each method, we measure the average end-to-end execution time required to produce a causal graph from a batch of time-windowed metrics, including the metric aggregation step (SR or FC) and the subsequent causal structure learning procedure. Because the absolute runtime depends on the implementation and environment, the figure reports the execution time ratio normalized by the proposed method (i.e., the proposed method is fixed to $1 \times$ and lower values indicate

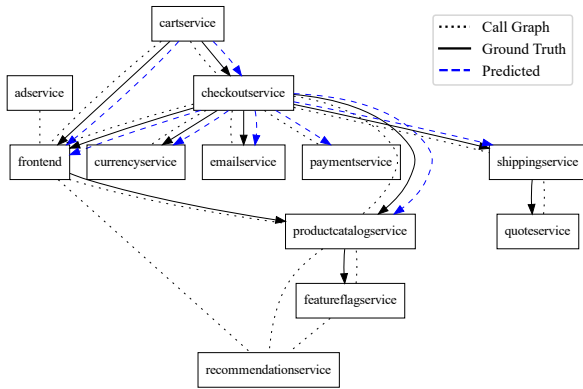


FIGURE 10: A case study: an output example of the proposed approach. Services are denoted by rectangles. The black dotted lines reflect the call graph. The black and blue solid lines denote the edges of the ground-truth and predicted causal graphs.

faster inference). The proposed method requires approximately 0.56 seconds to generate graphs for the evaluation batches. Given that each batch corresponds to a 180-second time window, this sub-second end-to-end runtime enables near real-time prediction for online troubleshooting pipelines. As shown in the figure, optimization-based structure learning methods such as NOTEARS and GOLEM incur substantially higher runtime, reflecting the iterative nature of their continuous optimization for each batch. In contrast, the proposed method generates causal graphs for a batch in a single forward pass, avoiding per-batch iterative optimization and enabling efficient simultaneous inference. Moreover, because metric aggregation and graph generation are integrated into the same inference pipeline, the proposed method provides a streamlined end-to-end workflow for efficient causal graph construction.

5) A Case Study

To further demonstrate the effectiveness of the proposed approach, we conduct a case study using a batch from Dataset A in Fig. 10. In this figure, rectangles represent services, while arrows denote causal relationships. Additionally, the dotted lines indicate service invocations observed during the experiment. The `cartservice` is a fault service, from which the fault propagates to other dependent services. Fig. 10 depicts the most representative example, where the proposed method successfully estimates the fault service and accurately identifies causal edges near the root cause. The fault location (i.e., the starting point of the causal graph) is correctly estimated with an accuracy of 0.976 in the evaluation data. However, the proposed method struggles to estimate causal edges between services located farther from the `cartservice`, confirming that fault propagation identification, particularly when distant from the root cause, is more challenging than fault detection itself.

VI. CONCLUSION AND DISCUSSION

A. CONCLUSION

This paper addressed the learning of causal structures of fault propagation in microservice-based systems, where noisy real-world monitoring telemetry often degrades the performance of existing causal discovery methods. To tackle this challenge, we proposed a semi-supervised framework that integrates causal graph generation with disentanglement learning under continuous structure learning. With a limited number of labeled graphs, the proposed model leverages abundant unlabeled metrics by jointly learning graph generation and structure regularization in a semi-supervised manner. Experiments on microservice-based system datasets showed that the proposed method outperforms representative baselines and that disentanglement improves robustness to out-of-distribution causal graphs.

B. LIMITATIONS

Our study has several limitations regarding supervision cost, scalability, and fault diversity. First, the framework assumes access to partial propagation-graph supervision for a subset of time windows. Operators tend to inspect only a small subset of representative time windows or sampled traces, which motivates our semi-supervised setting with abundant unlabeled metrics and a limited number of labeled graphs. Although semi-supervised learning reduces the number of required labels, creating high-quality graph annotations remains costly in practice, especially when traces are unavailable, heavily sampled, or noisy. Second, scalability is constrained by the graph-generation and acyclicity regularization costs. In particular, the acyclicity penalty operates on an adjacency matrix and can become a bottleneck as the number of services increases, while the sparsity of typical call graphs may mitigate the number of candidate edges in practice. Third, since our current experimental evaluation primarily covers single fault type injections, generalization to broader fault mechanisms requires further validation.

C. THREATS TO VALIDITY

Construct Validity: Our numerical evaluation relies on trace-derived reference graphs that are constructed by heuristic anomaly identification in span durations and by propagating anomalies along the observed call graph. Accordingly, the measured graph accuracy may be biased by trace availability, the anomaly thresholding strategy, and the assumption that dominant propagation during the injection period follows the call graph. Moreover, our evaluation metrics quantify structural similarity to the reference graphs, which may not fully capture the true causal mechanisms underlying the observed telemetry. Importantly, our framework can be applied to alternative supervision signals, such as human-annotated graphs or other forms of externally provided labels, when available.

Internal Validity: The compared baselines involve design and implementation choices (e.g., metric aggregation, call-graph constraints, and hyperparameter settings) that can affect the resulting graph quality. Moreover, unobserved con-

founders may violate causal sufficiency and bias the learned causal directions from passively observed telemetry. To mitigate this issue, we applied consistent constraints where possible and used publicly available implementations whenever available; otherwise, we implemented each baseline following the methodological details described in the original papers. Nevertheless, different implementations or hyperparameter effects may influence the absolute performance and the relative ranking.

External Validity: Our experiments are conducted on a finite set of datasets and injected fault settings. Therefore, the generalizability of the findings to other systems, workloads, observability configurations, and more diverse fault types remains to be further validated.

D. FUTURE WORK

Production incidents typically exhibit a large number of recurring patterns and a long tail of rare cases [24]. Our semi-supervised setting matches this operational reality, allowing it to leverage abundant unlabeled metrics while requiring only a small number of representative, partially labeled propagation graphs. Moreover, we will further examine the effectiveness of our framework under broader fault diversity and real-world incident conditions.

The dataset construction in this study relies on fault injection, benefiting from chaos engineering techniques [68]. A more convenient way to incorporate operator feedback (e.g., labeling only a few representative time windows or confirming a small set of edges) is needed to adapt the learned graphs to practical troubleshooting workflows. In addition, we plan to further reduce supervision by using few-shot learning [69] and related techniques. We will also investigate what forms of causal graphs are most actionable for operators, balancing interpretability with domain-knowledge cost and model efficiency. Interpretability should be balanced with domain-knowledge cost and model efficiency, and service-level graphs provide a practical granularity in this regard.

Finally, motivated by the internal validity threat of unobserved confounders, we plan to mitigate their impact by expanding observability beyond application-level metrics. Incorporating complementary telemetry can provide informative proxies for common causes, thereby reducing confounding effects and broadening fault coverage, especially for failure modes weakly expressed in a single modality. We will incorporate multimodal telemetry to improve anomaly understanding and diagnosis for broader failure modes [14], [70], [71].

REFERENCES

- [1] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "A survey on observability of distributed edge & container-based microservices," *IEEE Access*, vol. 10, pp. 86 904–86 919, 2022.
- [2] P. Jamshidi, C. Pahl, N. C. Mendonca, J. Lewis, and S. Tilkov, "Microservices: The journey so far and challenges ahead," *IEEE Softw.*, vol. 35, no. 3, pp. 24–35, May 2018.
- [3] G. Blinowski, A. Ojdowska, and A. Przybyłek, "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," *IEEE Access*, vol. 10, pp. 20 357–20 374, 2022.
- [4] P. D. Francesco, I. Malavolta, and P. Lago, "Research on architecting microservices: Trends, focus, and potential for industrial adoption," in *Proc. IEEE Int. Conf. Softw. Archit.*, Apr. 2017, pp. 21–30.
- [5] W. Saad, M. Bennis, and M. Chen, "A vision of 6G wireless systems: Applications, trends, technologies, and open research problems," *IEEE Netw.*, vol. 34, no. 3, pp. 134–142, May 2020.
- [6] S. D. A. Shah, M. A. Gregory, and S. Li, "Cloud-native network slicing using software defined networking based multi-access edge computing: A survey," *IEEE Access*, vol. 9, pp. 10 903–10 924, 2021.
- [7] H. Xu, W. Chen, N. Zhao, Z. Li, J. Bu, Z. Li, Y. Liu, Y. Zhao, D. Pei, Y. Feng, J. Chen, Z. Wang, and H. Qiao, "Unsupervised anomaly detection via variational auto-encoder for seasonal kpis in web applications," in *Proc. World Wide Web Conf.*, 2018, pp. 187–196.
- [8] H. Ren, B. Xu, Y. Wang, C. Yi, C. Huang, X. Kou, T. Xing, M. Yang, J. Tong, and Q. Zhang, "Time-series anomaly detection service at microsoft," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 3009–3017.
- [9] Z. He, P. Chen, X. Li, Y. Wang, G. Yu, C. Chen, X. Li, and Z. Zheng, "A spatiotemporal deep learning approach for unsupervised anomaly detection in cloud systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1705–1719, Apr. 2023.
- [10] S. Huang, Y. Liu, C. Fung, R. He, Y. Zhao, H. Yang, and Z. Luan, "HitAnomaly: Hierarchical transformers for anomaly detection in system log," *IEEE Trans. Netw. Serv. Manag.*, vol. 17, no. 4, pp. 2064–2076, Dec. 2020.
- [11] Y. Zuo, Y. Wu, G. Min, C. Huang, and K. Pei, "An intelligent anomaly detection scheme for micro-services architectures with temporal and spatial data analysis," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 2, pp. 548–561, Jun. 2020.
- [12] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, and C. Delimitrou, "Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices," in *Proc. Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2019, pp. 19–33.
- [13] L. Bao, C. Wu, X. Bu, N. Ren, and M. Shen, "Performance modeling and workflow scheduling of microservice-based applications in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 9, pp. 2114–2129, Sep. 2019.
- [14] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, D. Liu, Q. Xiang, and C. He, "Latent error prediction and fault localization for microservice applications by learning from system trace logs," in *Proc. ACM Joint Meet. Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2019, pp. 683–694.
- [15] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *IEEE/IFIP NOMS*, Apr. 2020, pp. 1–9.
- [16] A. Barredo Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Inf. Fusion*, vol. 58, pp. 82–115, Jun. 2020.
- [17] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, "Cloudranger: root cause identification for cloud native systems," in *Proc. IEEE/ACM Int. Symp. Clust. Cloud Grid Comput.*, 2018, pp. 492–502.
- [18] C. K. Assaad, E. Devijver, and E. Gaussier, "Survey and evaluation of causal discovery methods for time series," *J. Artif. Intell. Res.*, vol. 73, pp. 767–819, Feb. 2022.
- [19] J. Lin, P. Chen, and Z. Zheng, "Microscope: Pinpoint performance issues with causal graphs in micro-service environments," in *Proc. Int. Conf. Serv.-Oriented Comput.*, 2018, pp. 3–20.
- [20] S. Kobayashi, K. Otomo, K. Fukuda, and H. Esaki, "Mining causality of network events in log data," *IEEE Trans. Netw. Serv. Manag.*, vol. 15, no. 1, pp. 53–67, Mar. 2018.
- [21] A. Messenger, G. Parisi, I. Z. Kiss, R. Harper, P. Tee, and L. Berthouze, "Inferring functional connectivity from time-series of events in large scale network deployments," *IEEE Trans. Netw. Serv. Manag.*, vol. 16, no. 3, pp. 857–870, Sep. 2019.
- [22] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, and D. Pei, "Causal inference-based root cause analysis for online service systems with intervention recognition," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, vol. 53, Aug. 2022, pp. 3230–3240.
- [23] A. Ikram, S. Chakraborty, S. Mitra, S. K. Saini, S. Bagchi, and M. Kocaoglu, "Root cause analysis of failures in microservices through causal discovery," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022.
- [24] Z. Li, N. Zhao, M. Li, X. Lu, L. Wang, D. Chang, X. Nie, L. Cao, W. Zhang, K. Sui, Y. Wang, X. Du, G. Duan, and D. Pei, "Actionable and interpretable fault localization for recurring failures in online service systems," in *Proc.*

- ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng., 2022, pp. 996–1008.
- [25] S. Chakraborty, S. Garg, S. Agarwal, A. Chauhan, and S. K. Saini, “CausalIL: Causal Graph for Instance Level Microservice Data,” in *Proc. ACM Web Conf.*, vol. 11, Apr. 2023, pp. 2905–2915.
- [26] D. Wang, Z. Chen, Y. Fu, Y. Liu, and H. Chen, “Incremental causal graph learning for online root cause analysis,” in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, Aug. 2023, pp. 2269–2278.
- [27] R. Xin, P. Chen, and Z. Zhao, “Causalrca: Causal inference based precise fine-grained root cause localization for microservice applications,” *J. Syst. Softw.*, vol. 203, no. C, Sep. 2023.
- [28] T. Li, L. Wang, D. Peng, J. Liao, L. Liu, and Z. Liu, “Causal discovery evaluation framework in the absence of ground-truth causal graph,” *IEEE Access*, vol. 12, pp. 136 502–136 514, 2024.
- [29] K. Göbler, T. Windisch, M. Drton, T. Pichynski, M. Roth, and S. Sonntag, “causalAssembly: Generating Realistic Production Data for Benchmarking Causal Discovery,” in *Proc. Causal Learn. Reason.*, Mar. 2024, pp. 609–642.
- [30] L. Pham, H. Ha, and H. Zhang, “Root cause analysis for microservice system based on causal inference: How far are we?” in *Proc. IEEE/ACM Int. Conf. Autom. Softw. Eng.*, Oct. 2024, pp. 706–715.
- [31] M. Mathieu, J. Zhao, P. Sprechmann, A. Ramesh, and Y. LeCun, “Disentangling factors of variation in deep representations using adversarial training,” in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Dec. 2016, pp. 5047–5055.
- [32] R. Cai, Z. Li, P. Wei, J. Qiao, K. Zhang, and Z. Hao, “Learning disentangled semantic representation for domain adaptation,” in *Proc. Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 2060–2066.
- [33] S. Lee, S. Cho, and S. Im, “DRANet: Disentangling representation and adaptation networks for unsupervised cross-domain adaptation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2021, pp. 15 247–15 256.
- [34] M. Kim, M. Cho, and S. Lee, “Feature disentanglement learning with switching and aggregation for video-based person re-identification,” in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, Jan. 2023, pp. 1603–1612.
- [35] P. Spirtes, C. Glymour, and R. Scheines, *Causation, prediction, and search*, 2nd ed. Springer, Dec 2012.
- [36] H. Zhang, S. Zhou, J. Guan, and J. I. Huan, “Measuring conditional independence by independent residuals for causal discovery,” *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 5, pp. 1–19, Sep. 2019.
- [37] X. Guo, Y. Wang, X. Huang, S. Yang, and K. Yu, “Bootstrap-based causal structure learning,” in *Proc. ACM Int. Conf. Inf. Knowl. Manag.*, Oct. 2022, pp. 656–665.
- [38] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing, “Dags with no tears: continuous optimization for structure learning,” in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 9492–9503.
- [39] Y. Yu, J. Chen, T. Gao, and M. Yu, “DAG-GNN: DAG Structure Learning with Graph Neural Networks,” in *Proc. Int. Conf. Mach. Learn.*, May 2019, pp. 7154–7163.
- [40] S. Lachapelle, P. Brouillard, T. Deleu, and S. Lacoste-Julien, “Gradient-Based Neural DAG Learning,” in *Proc. Int. Conf. Learn. Representations*, Sep. 2019.
- [41] R. Pamfil, N. Sriwattanaworachai, S. Desai, P. Pilgerstorfer, K. Georgatzis, P. Beaumont, and B. Aragam, “Dynotears: Structure learning from time-series data,” in *Proc. Int. Conf. Artif. Intell. Stat.*, vol. 108, 26–28 Aug 2020, pp. 1595–1605.
- [42] I. Ng, A. Ghassami, and K. Zhang, “On the role of sparsity and DAG constraints for learning linear DAGs,” in *Proc. Int. Conf. Neural Inf. Process. Syst.*, no. Article 1506, Dec. 2020, pp. 17 943–17 954.
- [43] A. Bellot, K. Branson, and M. van der Schaar, “Neural graphical modelling in continuous-time: consistency guarantees and algorithms,” in *Proc. Int. Conf. Learn. Representations*, Oct. 2021.
- [44] Y. He, P. Cui, Z. Shen, R. Xu, F. Liu, and Y. Jiang, “DARING: Differentiable causal discovery with residual independence,” in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, Aug. 2021, pp. 596–605.
- [45] H. Shan, Y. Chen, H. Liu, Y. Zhang, X. Xiao, X. He, M. Li, and W. Ding, “ ϵ -diagnosis: Unsupervised and real-time diagnosis of small- window long-tail latency in large-scale microservice platforms,” in *Proc. World Wide Web Conf.*, 2019, pp. 3215–3222.
- [46] L. Pham, H. Ha, and H. Zhang, “Baro: Robust root cause analysis for microservices via multivariate bayesian online change point detection,” *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024.
- [47] D. Liu, C. He, X. Peng, F. Lin, C. Zhang, S. Gong, Z. Li, J. Ou, and Z. Wu, “Microhecl: high-efficient root cause localization in large-scale microservice systems,” in *Proc. IEEE/ACM Int. Conf. Softw. Eng.: Softw. Eng. Pract.*, 2021, pp. 338–347.
- [48] Y. Meng, S. Zhang, Y. Sun, R. Zhang, Z. Hu, Y. Zhang, C. Jia, Z. Wang, and D. Pei, “Localizing failure root causes in a microservice through causality inference,” in *Proc. IEEE/ACM Int. Symp. Qual. Serv.*, Jun. 2020, pp. 1–10.
- [49] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou, “Sage: practical and scalable ML-driven performance debugging in microservices,” in *Proc. ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, Apr. 2021, pp. 135–151.
- [50] X. Yang, Z. Song, I. King, and Z. Xu, “A survey on deep semi-supervised learning,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 8934–8954, Sep. 2023.
- [51] S. Xu, Y. Qian, and R. Q. Hu, “A semi-supervised learning approach for network anomaly detection in fog computing,” in *Proc. IEEE Int. Conf. Commun.*, May 2019, pp. 1–6.
- [52] L. Yang, J. Chen, Z. Wang, W. Wang, J. Jiang, X. Dong, and W. Zhang, “Semi-supervised log-based anomaly detection via probabilistic label estimation,” in *Proc. IEEE/ACM Int. Conf. Softw. Eng.*, May 2021, pp. 1448–1460.
- [53] W. Meng, Y. Liu, S. Zhang, F. Zaiter, Y. Zhang, Y. Huang, Z. Yu, Y. Zhang, L. Song, M. Zhang, and D. Pei, “LogClass: Anomalous log identification and classification with partial labels,” *IEEE Trans. Netw. Serv. Manag.*, vol. 18, no. 2, pp. 1870–1884, Jun. 2021.
- [54] K. Zhang, C. Zhang, X. Peng, and C. Sha, “PUTraceAD: Trace anomaly detection with partial labels based on GNN and PU learning,” in *Proc. IEEE Int. Symp. Softw. Rel. Eng.*, Oct. 2022, pp. 239–250.
- [55] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, and D. Pei, “Practical root cause localization for microservice systems via trace analysis,” in *Proc. IEEE/ACM Int. Symp. Qual. Serv.*, Jun. 2021, pp. 1–10.
- [56] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, and X. Li, “Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments,” in *Proc. ACM Web Conf.*, 2021, pp. 3087–3098.
- [57] J. Pearl, *Causality: Models, reasoning, and inference*, 2nd ed. Cambridge University Press, Sep. 2009.
- [58] J. Peters, J. M. Mooij, D. Janzing, and B. Schölkopf, “Causal discovery with continuous additive noise models,” *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 2009–2053, Jan. 2014.
- [59] A. Vaswani, N. M. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is All you Need,” *Proc. Int. Conf. Neural Inf. Process. Syst.*, vol. 30, pp. 5998–6008, Jun. 2017.
- [60] S. Shin, Y. He, M. Vankadari, T.-Y. Cheng, Q. Xie, A. Markham, and N. Trigoni, “Towards Learning Group-Equivariant Features for Domain Adaptive 3D Detection,” in *Proc. Adv. Neural Inf. Process. Syst.* Vancouver, BC, Canada: Neural Information Processing Systems Foundation, Inc., Dec. 2024, pp. 18 019–18 035.
- [61] S. Tan, B. Gong, B. Ji, and Y. Pan, “FixTalk: Taming Identity Leakage for High-Quality Talking Head Generation in Extreme Cases,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2025, pp. 24–36.
- [62] L. Pham, H. Zhang, H. Ha, F. Salim, and X. Zhang, “RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data,” in *Proc. ACM Web Conf.* New York, NY, USA: ACM, May 2025, pp. 777–780.
- [63] X. Zhou, X. Peng, T. Xie, J. Sun, C. Xu, C. Ji, and W. Zhao, “Benchmarking microservice systems for software engineering research,” in *Proc. Int. Conf. Softw. Eng.*, 2018, pp. 323–324.
- [64] I. Tsamardinos, L. E. Brown, and C. F. Aliferis, “The max-min hill-climbing Bayesian network structure learning algorithm,” *Mach. Learn.*, vol. 65, no. 1, pp. 31–78, Oct. 2006.
- [65] T. Qin, T.-Z. Wang, and Z.-H. Zhou, “Learning causal structure on mixed data with tree-structured functional models,” in *Proc. SIAM Int. Conf. Data Mining*, Jan. 2023, pp. 613–621.
- [66] A. B. Barrett, L. Barnett, and A. K. Seth, “Multivariate Granger causality and generalized variance,” *Phys. Rev. E Stat. Nonlin. Soft Matter Phys.*, vol. 81, no. 4 Pt 1, p. 041907, Apr. 2010.
- [67] D. C. Liu and J. Nocedal, “On the limited memory BFGS method for large scale optimization,” *Math. Program.*, vol. 45, no. 1-3, pp. 503–528, Aug. 1989.
- [68] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, “Chaos Engineering,” *IEEE Softw.*, vol. 33, no. 3, pp. 35–41, May 2016.

- [69] S. Aoki, K. Shiimoto, and C. L. Eng, "Few-shot learning and self-training for eNodeB log analysis for service-level assurance in LTE networks," *IEEE Trans. Netw. Serv. Manag.*, vol. 17, no. 4, pp. 2077–2089, Dec. 2020.
- [70] L. Zheng, Z. Chen, J. He, and H. Chen, "MULAN: Multi-modal causal structure learning and root cause analysis for microservice systems," in *Proc. ACM Web Conf.*, vol. 229. New York, NY, USA: ACM, May 2024, pp. 4107–4116.
- [71] L. Tao, S. Zhang, Z. Jia, J. Sun, M. Ma, Z. Li, Y. Sun, C. Yang, Y. Zhang, and D. Pei, "Giving every modality a voice in microservice failure diagnosis via multimodal adaptive optimization," in *Proc. IEEE/ACM Int. Conf. Autom. Softw. Eng.* New York, NY, USA: ACM, Oct. 2024, pp. 1107–1119.



CHAO WU received her bachelor's degree in engineering from Zhejiang University in 2009, master's degree in engineering from Waseda University in 2013, and Ph.D. degree from Tsukuba University in 2023. In 2014, she joined NTT Access Network Service Systems Laboratories, where she has been researching and developing management mechanisms for telecommunications. Her research interests include intent-based management, cloud, and virtualization.



NOBUKAZU FUKUDA received his bachelor's degree in engineering from Kyoto University in 2018 and master's degree in information science and technology from Tokyo University in 2020. He is currently a research engineer with NTT Access Network Service Systems Laboratories. His research interests include fault analysis of distributed systems and network operation automation. He is a member of IEICE. He received the Young Researcher's Award (IEICE), Japan, in 2025.



SHINGO HORIUCHI received his B.E. and M.E. degrees in engineering from the University of Tokyo in 1999 and 2001. In 2001, he joined NTT Access Network Service Systems Laboratories, where he has been researching and developing access network operation systems. Since 2014, he has been engaged in the standardization work for operation support systems in TMForum as a member of the Open Digital Architecture Project. He is a member of IEICE.



HARUHISA NOZUE received his M.S. degree in mathematical sciences from Nagoya University, Aichi, Japan, in 2003. He joined NTT Access Network Service Systems Laboratories in 2003, where he is a senior research engineer of the Access Network Operation Project and is engaged in the research and development of operation support systems of access networks.



HARUO OISHI received his B.E and M.E. degrees in electrical engineering from Tokyo Institute of Technology in 1995 and 1997, and his Ph.D. degree in computer science and communications engineering from Waseda University in 2022. Since joining NTT Corporation in 1997, he has engaged in R&D on educational technology, mobile PC middleware technology, business Ethernet service management system development, network failure estimation technology, and business navigation technology. He presently works at NTT Access Network Service System Laboratories as the leader of the Access Operation Project Collaborative Operation Technology Group. He is a senior member of IEICE and the chair of the Technical Committee on Information and Communication Management (ICM).



KENICHI TAYAMA received his B.E. and M.E. degrees in electrical engineering from the University of Electro-Communications, Tokyo, in 1993 and 1995. He joined NTT Optical Network Systems Laboratories in 1995. He also worked at NTT EAST's IT Innovation Department and NTT-ME's Network Operation Center, where he researched and developed network operations support systems. He is a member of IEICE.

...